

Lab Assignment 1 :

Objective : Get familiar with the working of a production quality open source compiler : gcc. Acquire ability to view and understand the internal dumps produced which shall be used as benchmark for the experiments that you will conduct henceforth.

Setting up for the Session :

- Use putty and the login credentials given by the technical staff to log into the Unix/Linux server.
- Read the accompanying document, Lab-conduct-potocol.pdf". Learn a few useful options that can be used when compiling a source program with gcc and their implications which are given in the Appendix.

Problem 1 Consider the simple C program, "testprog.c" given to you. and compile through the options listed below.

(a) Compile the C program using : `$ gcc -v testprog.c 2> gcc-output -o testunopt`

Find out all the files created by the execution of the above command. Open the file, "gcc-output" and examine the contents. Interpret the contents to determine information about , i) versions of the gcc and Linux/Unix, ii) number and names of the executables used, and iii) identify the names of as many cousins of the C compiler as you can.

(b) run the executable, "testunopt" under "time" command and save the output in a file for examination later.

(c) Use the switch, "-o" to give the executable file a name chosen by you (instead of "a.out") and repeat step (b) above with the new name.

Problem 2. Generate the various intermediate dumps generated by the compiler and compare them with the given source, particularly the compiler dumps after the 5th and 6th passes named in ".original", and ".gimple". Save your findings in a file.

Problem 3. Use the "-O2" optimization switch to compile "testprog.c and name the executable, say, "testopt". Run the executable, "testopt" under "time" command. Compare the results obtained now, both the running time and the output values with similar output obtained in Problem 1(b). Report the findings and speculate about the differences, if any. Can you read the source code of "testprog.c" and manually make changes so that the resulting source is more efficient than the given one.

Problem 4. Generate annotated assembly codes for "testprog.c" without and with using the switch "-O2". Compare the sizes and contents of the two assembly codes and make observations about which appears to be more efficient.

```
// source program testprog.c
#include<stdio.h>
#include <stdlib.h>
int a[50000], b[50000];
int main()
{ for ( int k = 0; k < 100; k++)
  { for (int i = 0 ; i < 10000; i++)
    { a[i] = 5*i;
      for (int j = 0; j < 10000; j++)
        { b[j] = j*j;}
      a[i] = a[i]+ b[i];
    }
  }

  int index1, index2;
  index1 = rand()%10000;
  index2 = rand()%10000;
  printf(" %d %d %d %d \n", index1, a[index1], index2, b[index2]);
  return 0;
}
```

APPENDIX : Basic Switches of gcc

- 1 **Use of the compiler switch -c** : this switch informs the compiler that the user is only interested in compiling the submitted program but does not want the executable to be generated.
- 2 **Use of the compiler switch -o** : this switch (letter o in small case) informs the compiler that the user is interested in giving her/his own name to the generated executable file and override the default name “a.out” used by the compiler. As a good practice, it is better to give a name to each executable program, instead of using the default name, because the latter gets overwritten after every compilation, while the former remains saved with us.
- 3 **Use of compiler switch -S** : assembly code is generated by the compiler and saved in the file “testprog.s”. The assembly code generated directly by this switch is difficult to read.

\$ gcc -S testprog.c will generate assembly code in testprog.s

A better human readable assembly code (annotated assembly) is generated using an additional switch, “-fverbose-asm”, for example \$ gcc -S -fverbose-asm testprog.c will generate annotated assembly in testprog.s

4 **Use of the verbose switch “-v” of gcc**

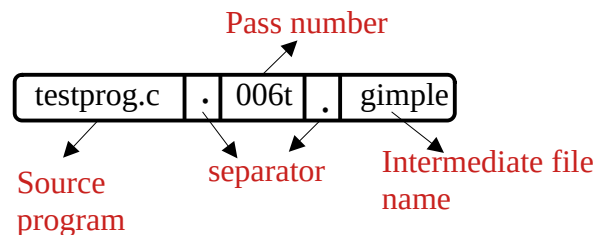
In contrast to the common misconception that gcc (gnu C compiler) is the compiler proper, it is in reality a shell script that calls various tools for performing compilation and related tasks. The switch “-v” can be used to know the internal structure of gcc. The switch “-v” usually stands for verbose description.

5 Generating intermediate compiler dumps in gcc

In order to generate the intermediate dumps produced by gcc, we use the switches given below. Note that there may be small variations in the dumps and their formats depending on the gcc version used. The verbose dump gives the version of gcc and version of ubuntu that is used in my laptop. Earlier or later versions may give slightly different dumps, **gcc version 11.4.0 (Ubuntu 11.4.0-2ubuntu1~20.04)**. Earlier or later versions would give slightly different dumps. \$ gcc --version will inform the version of the compiler.

Observations : Let the command be \$ gcc -fdump-tree-all testprog.c

The internal files that are generated are named as “labprog1.c.dddt.name”, that is the source program name suffixed with dot (.) followed by 3 digits followed by t; which in turn is followed by a file-name. The naming convention used is the following.



6 Understanding the Intermediate Compiler Dumps

In the figure above, the file “gimple” is the dump produced by gcc after the 6th pass. Also it appears that gcc performed a large number of passes and the result are saved in an appropriately named intermediate dump file.

- It may be noted that gimple is the name of intermediate language used by gcc (other compilers may use AST, RTL, etc instead). The result of the 6th pass is therefore the source program translated into gimple language.
- The other passes use appropriate names to indicate the action performed by the pass. For example, the 5th pass gives the original program after processing; the 15th pass generates the control flow graph (cfg); the 23rd pass translated the intermediate code to static single assignment (ssa); 244th pass produces an optimized version, and so on.

File i/o using the Shell

A small digression for those who are NOT familiar with shell programming. There are 3 operators used by the shell for i/o redirection (redirecting the standard input, standard output and standard error files to other files chosen by a user rather than their default binding to device files.

Operator	Usage	Semantics
<	\$./a.out < file1	The input required by a.out will be read from “file1” when the program reads using scanf() or equivalent functions.
> or 1>	\$./a.out > file2 \$./a.out 1> file2	The output produced by a.out, using printf() or equivalent, will be written to “file2”
2>	\$./a.out 2> file3	The error messages produced by a.out, using the standard error channel, will be written to “file3”
>> or 2>>	\$./a.out >> file4 \$./a.out 2>> file5	Same effect as > or 2> except that the data is appended to the files instead of overwriting.